

Wifite

The Reaver guide established the key distinction: Wifite introduces no new underlying attack beyond what the Aircrack-ng and Reaver guides already cover — its entire value is **orchestration**, automatically deciding which of those two techniques (or several others) to attempt against each nearby target, and sequencing the many manual steps each requires into a largely unattended workflow. Worth locking in the distinction immediately: **Part 1 covers Wifite's automation model — how it selects an attack strategy per target and what it's actually doing under the hood at each stage — Part 2 covers practical usage, configuration options, and where automation's convenience trades off against the manual control the underlying tools offer.**

1. Why an Orchestration Layer Is Useful — The Foundation

A real-world wireless assessment against a single target already requires chaining several manual steps across the Aircrack-ng suite (monitor mode → scan → targeted capture → optional deauthentication → crack) or Reaver (WPS scan → PIN attack → timing adjustment). Against **multiple nearby networks** in a single session, this manual sequencing multiplies — re-running the same multi-tool workflow, with slightly different parameters, once per target.

Stage	Manual Workflow
1. Discovery	Identify the target using <code>airodump-ng</code> or <code>wash</code> .
2. Classification	Determine whether WPS is enabled.
3. Attack Selection	WPS enabled → Reaver; WPS disabled → Aircrack-ng workflow.
4. Session Management	Manage timing, lockout considerations, and tool-specific parameters.
5. Results	Record the result and move to the next authorized target.

The single fact this guide is built around — Wifite automates exactly this decision tree and re-execution loop, selecting an appropriate attack per target based on its detected configuration (WPS status, encryption type), running the underlying Aircrack-ng/Reaver components itself, and cycling through multiple targets in a single supervised session — without

introducing any cryptographic or protocol-level technique the underlying tools don't already provide individually.

PART 1: Automation Model

2. Target Discovery and Automatic Classification

```
wifite
```

When launched, Wifite performs its own wireless discovery and presents nearby networks with information such as:

- **Encryption type:** WEP, WPA, WPA2, WPA3, or Open
- **WPS status:** Enabled, locked, or not detected
- **Signal strength**
- **Client activity**

This classification drives Wifite's automatic attack-selection logic and consolidates information that would otherwise be gathered separately with

`airodump-ng` and `wash`.

3. Attack Selection Logic

Detected Configuration	Default Approach
WEP	Aircrack-ng-style statistical IV attack with replay acceleration
WPA/WPA2 + WPS enabled	Reaver-style WPS PIN attack attempted first
WPA/WPA2 + WPS disabled/failed	Aircrack-ng-style handshake capture, optional deauthentication, and offline dictionary attack
WPA3	Generally skipped or flagged because these techniques do not provide the same attack path against SAE

This priority ordering (WPS-based attack attempted before falling back to handshake capture) directly reflects the practical reality covered in the Reaver guide — when WPS is viable, it often provides a faster, passphrase-strength-independent path to the same outcome.

4. Session Management Across Multiple Targets

Given a list of nearby networks, Wifite proceeds through each one in sequence — attempting its selected strategy, recording success or failure, and moving to the next target automatically rather than requiring an operator to manually restart the entire workflow per network. This is Wifite's single largest practical advantage over manually invoking Aircrack-ng and Reaver: **consistency and throughput** across many targets in one supervised session, rather than any new capability.

PART 2: Practical Usage & Configuration

5. Basic Invocation

```
wifite
```

Run without arguments, Wifite scans, presents discovered targets interactively, and allows the operator to select which to attack (individually or via an "attack all" option) — a reasonable default for a straightforward assessment session.

6. Scoping to a Specific Target

```
wifite --bssid AA:BB:CC:11:22:33
```

```
--bssid  → restrict the session to a single specific target,
           equivalent in intent to Aircrack-ng's targeted
           capture (-c/--bssid flags) or Reaver's -b flag,
           useful when the assessment's scope is a known,
           specific network rather than an open survey
```

7. Controlling Which Underlying Techniques Are Attempted

```
wifite --no-wps          # skip WPS/Reaver-based attacks entirely
wifite --wps-only        # attempt ONLY WPS-based attacks, skipping
                           handshake capture entirely
wifite --dict wordlist.txt # specify the wordlist used for any
                           offline WPA/WPA2 handshake cracking
                           attempt (per the Aircrack-ng guide's
                           Section 8 wordlist-quality dependency)
```

These flags matter because Wifite's default automatic selection (Section 3) isn't always the desired approach for a given assessment — for instance, an assessor specifically evaluating WPS exposure across many targets would use `--wps-only` rather than allowing Wifite to also spend time on handshake-based attempts.

8. Adjusting Timing Behavior

```
wifite --wps-timeout 300 # cap time spent on a WPS attack per
                        target before moving on
wifite --dict-timeout 600 # cap time spent on an offline dictionary
                        crack attempt per target
```

Since a single unproductive attempt (e.g., a WPS target with an aggressive lockout policy, per the Reaver guide's Section 3) could otherwise consume disproportionate session time, timeout controls keep a multi-target session moving rather than stalling indefinitely on one difficult target.

9. Where Automation's Convenience Has a Cost

Area	Wifite Automation	Direct Tool Usage
Timing	Uses automated/default behavior	Allows finer target-specific tuning
Debugging	Abstracts several underlying steps	Provides more direct Aircrack-ng/Reaver output
Difficult targets	Convenient for broad coverage	Better suited to patient, manual tuning
Primary strength	Breadth and consistency	Depth and control

This is the direct, practical version of the general trade-off between automation and manual control: Wifite is the right tool for breadth (many targets, consistent baseline effort each) while direct Aircrack-ng/Reaver usage remains the right tool for depth (one specific, high-value target warranting careful manual tuning).

10. Testing Methodology

1. Confirm authorization

Verify that every network potentially included in the Wifite session is within the approved scope.

2. Perform an initial scan

Review Wifite's automatic classification before allowing any attack attempts.

3. Restrict the target scope

Use target-specific options such as `--bssid` to limit testing to explicitly authorized networks.

4. Select the assessment technique

Use technique-restriction options such as `--wps-only` or `--no-wps` when they match the assessment objective.

5. Review automated results

Monitor the session and identify targets where Wifite's default behavior is insufficient.

6. Fall back to manual tools when necessary

For a difficult or high-value target, use Aircrack-ng or Reaver directly to obtain finer control over timing, retries, and troubleshooting.

11. Legal & Ethical Note

Wifite should only be used against wireless networks for which you have **explicit authorization**.

Because Wifite can automatically discover and attempt attacks against multiple nearby networks, scope control is especially important. A network appearing in a scan does **not** imply permission to test it.

Best practices:

- Test only explicitly authorized networks.
- Define the assessment scope before starting a session.
- Use target-specific restrictions where possible.
- Avoid broad "attack all" behavior in environments containing networks outside the engagement scope.
- Document the permitted techniques and testing window.

12. Prevention

Wifite does not introduce a new underlying wireless vulnerability. Its prevention measures therefore correspond to the techniques it automates.

Wireless Security Controls

- **Use WPA3 where supported.**

This addresses the WPA/WPA2 handshake-cracking path that Wifite may otherwise attempt.

- **Disable WPS when it is not required.**

This removes the WPS attack path that Wifite prioritizes when WPS is available.

- **Use strong WPA2 passphrases.**

Strong, unique passphrases reduce the effectiveness of offline dictionary attacks against captured WPA/WPA2 handshakes.

- **Deploy WIDS/WIPS monitoring.**

Wireless monitoring can help detect abnormal deauthentication activity and repeated WPS authentication attempts.

- **Keep wireless infrastructure updated.**

Firmware and security updates can improve protection against known wireless implementation weaknesses.

13. Quick Reference Cheat Sheet

Basic Run

```
wifite
```

Scope to a Specific Target

```
wifite --bssid <target>
```

Restrict Techniques

```
wifite --no-wps
wifite --wps-only
wifite --dict wordlist.txt
```

Timing Controls

```
wifite --wps-timeout 300
wifite --dict-timeout 600
```

Default Attack-Selection Logic

Target Configuration	Wifite's General Approach
WEP	Statistical IV attack
WPA/WPA2 + WPS enabled	Reaver-style WPS assessment first
WPA/WPA2 + WPS disabled	Aircrack-ng-style handshake capture and dictionary attack
WPA3	Generally skipped for these attack paths

Key Takeaways

- **Wifite is an orchestration layer**, not a new cryptographic attack.
- Its primary benefit is **automated, consistent coverage across multiple targets**.
- WPS availability influences whether Wifite prioritizes a Reaver-style assessment.
- WPA/WPA2 targets without a viable WPS path may move to an Aircrack-ng-style workflow.
- Direct Aircrack-ng or Reaver usage provides **greater manual control** for difficult individual targets.